Heart Disease Risk Prediction

An End-to-End MLOps Pipeline: Data Engineering, Model Development, Experiment Tracking, CI/CD, Containerization, Kubernetes Deployment & Monitoring

AIMLCZG523 — Machine Learning Operations (MLOps)
Assignment 1 — End-to-End ML Model Development, CI/CD, and Production Deployment

Roll Number: 2025AE05681
Name: Sowjanya | **Programme:** BITS Pilani — WILP

Dataset: Heart Disease UCI Dataset (Cleveland subset), UCI Machine Learning Repository
Code Repository: https://github.com/SaiSowjanyaVadapalli/heart-disease-mlops

---

Submitted as part of the MLOps Experimental Learning Assignment

Table of Contents

1. Introduction, Problem Statement & Objectives
2. Environment Setup
3. Data Acquisition
4. Exploratory Data Analysis (EDA)
5. Feature Engineering
6. Model Development & Hyperparameter Tuning
7. Model Evaluation and Results
8. Experiment Tracking with MLflow & Model Packaging
9. Automated Testing Strategy
10. CI/CD Pipeline Design (GitHub Actions)
11. Model Containerization with Docker
12. Production Deployment to Kubernetes
13. Monitoring and Logging
14. System Architecture
15. Key Design Decisions
16. Challenges Faced and Resolutions
17. Conclusion, Future Work & Repository

1. Introduction, Problem Statement & Objectives

This assignment uses heart disease risk classification as a vehicle for demonstrating end-to-end MLOps practice: not just an accurate classifier, but every piece of infrastructure and automation that takes a trained model from experiment to a production-grade, monitored service. The task is binary classification — given 13 clinical attributes (age, sex, chest pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG result, max heart rate, exercise-induced angina, ST depression, ST slope, number of major vessels, and thalassemia result) — predict presence or absence of heart disease, using the Cleveland Clinic subset of the UCI Heart Disease dataset, the most complete and widely used of the UCI collection's four hospital subsets.

The assignment brief specifies nine graded components: data acquisition & EDA, feature engineering & modelling, experiment tracking, model packaging, CI/CD & testing, containerization, deployment, monitoring, and documentation. This report walks through each in implementation order, explaining not just *what* was built but *why*, since a guiding principle throughout was that any step that can only be run once, by one person, on one machine, is not really part of an MLOps pipeline — every stage here is re-runnable by anyone, on a clean machine, with the same result.

2. Environment Setup

The project targets Python 3.11 in production (matched by the Docker image and CI runners), with every dependency pinned to an exact version in `requirements.txt` so development, CI, and the container all use identical library versions — important because scikit-learn can produce numerically different artifacts across minor versions. From a clean checkout:

```
git clone https://github.com/SaiSowjanyaVadapalli/heart-disease-mlops
cd heart-disease-mlops && python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

No system dependencies beyond Python are needed for the ML/API portions. Docker and a local Kubernetes cluster are needed only for Sections 11-12; a GitHub Codespaces alternative needing nothing installed locally is documented in `k8s/README.md` and discussed in Section 12.

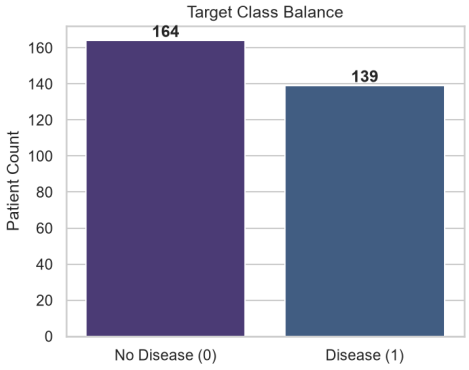
3. Data Acquisition

`scripts/download_data.py` downloads `processed.cleveland.data` directly from `archive.ics.uci.edu` rather than using a pre-cleaned third-party CSV mirror — a deliberate choice, since it exercises the raw-data-handling skill the assignment assesses and keeps this pipeline distinct from the common mirror most tutorials use. The script: (1) assigns the 14 standard column names to the header-less raw file; (2) converts the literal `'?'` missing-value sentinel (found only in `ca` and `thal`, 6 rows total) to a proper NaN via pandas; (3) binarizes the original 0-4 severity column `num` into a binary `target` (0 = no disease, 1 = disease present), as the assignment requires; and (4) falls back to a mirrored copy of the same data if UCI is unreachable, so a transient network

issue never hard-fails the pipeline. The 6 missing rows are not dropped; they are imputed inside the modelling pipeline (Section 5) so no observation is discarded and the imputation strategy stays reproducible.

4. Exploratory Data Analysis (EDA)

EDA is a standalone, re-runnable script (`notebooks/01_eda.py`) rather than a notebook, so CI can regenerate the exact same figures with no manual "run all cells" step. It produces five figures into `reports/figures/`.



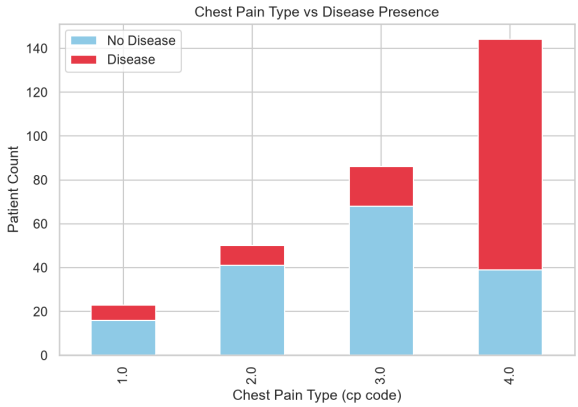


Figure 1 (left). Target class balance. *Figure 2 (right). Chest pain type vs. diagnosis.*

Class balance (Figure 1): of 303 patients, 165 (54.1%) have no disease and 138 (45.9%) have disease present — close enough to balanced that accuracy remains meaningful, so there was no need for class-imbalance mitigation such as SMOTE or class-weighted loss, both of which add complexity and their own failure modes.

Chest pain vs. diagnosis (Figure 2): this is the most clinically counter-intuitive finding in the dataset — "asymptomatic" chest pain (code 4) is the category most strongly associated with disease presence, while "typical anginal" pain (code 1) is more common among disease-negative patients. This is in fact well documented in cardiology (asymptomatic presentation delays diagnosis), and it is the reason `cp` is one-hot encoded as a nominal category rather than treated as an ordinal severity scale — the numeric codes 1-4 do not represent increasing risk in a straight line.

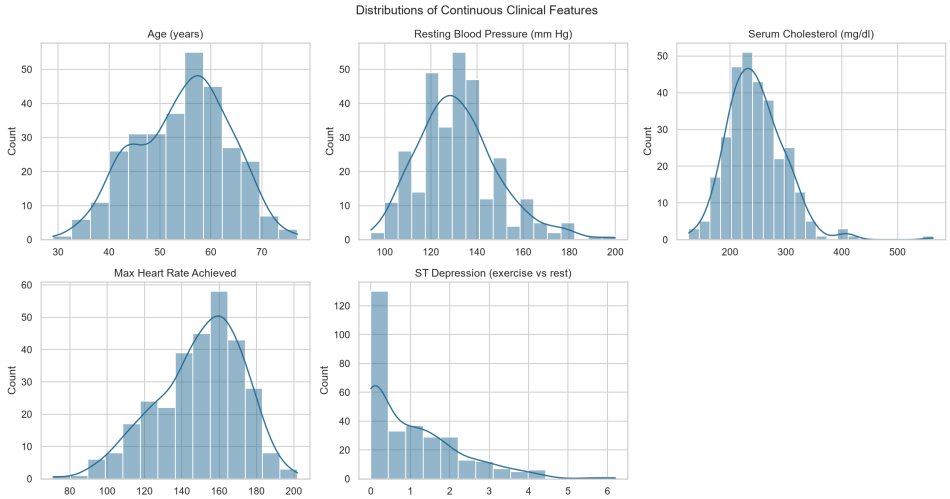


Figure 3. Distributions of the five continuous clinical features.

Age, resting blood pressure, and max heart rate are close to normally distributed; cholesterol and ST depression (oldpeak) are both visibly right-skewed with extreme outliers (one cholesterol reading exceeds 550 mg/dl against a median around 240). This directly motivated two later decisions: **median** (not mean) imputation for missing numeric values, since the mean is far more sensitive to these outliers, and a **standard scaler**, since Logistic Regression's regularization penalty is only meaningful once every feature is on a comparable scale.

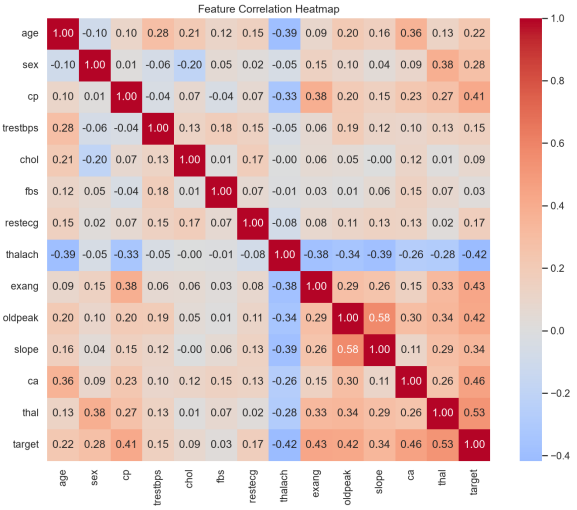


Figure 4. Pairwise correlation heatmap.

The features most correlated with the target are thalassemia result (thal, r≈0.53), number of major vessels colored by fluoroscopy (ca, r≈0.46), exercise-induced angina (exang, r≈0.43), ST depression (oldpeak, r≈0.43), and max heart rate (thalach, r≈-0.42) — all established cardiology risk indicators, a useful sanity check that the data and target encoding agree with domain knowledge. No predictor pair showed problematic multicollinearity (largest magnitude well under 0.8), so no features were dropped on that basis.

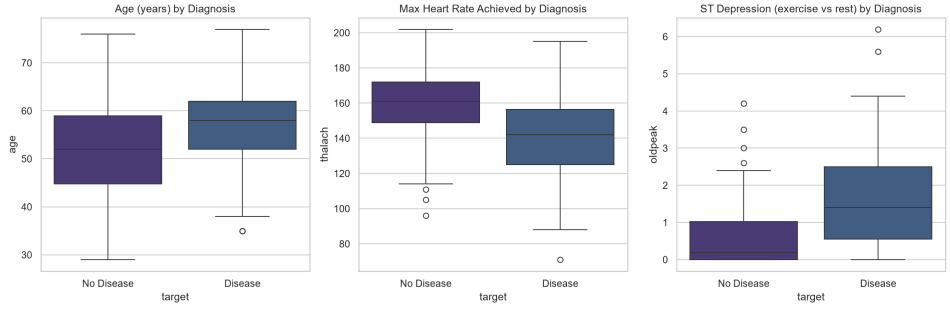


Figure 5. Age, max heart rate, and ST depression, split by diagnosis.

Disease-positive patients skew older, achieve a lower max heart rate under exercise, and show markedly greater ST depression — all textbook findings for reduced cardiovascular exercise tolerance. The substantial overlap in each panel is itself informative: no single feature is anywhere close to a sufficient predictor alone, which justifies a multivariate model over any simple

single-threshold clinical rule.

5. Feature Engineering

Implemented once in `src/data_processing.py` as a single scikit-learn `ColumnTransformer`, rather than ad-hoc pandas logic duplicated between training and serving code — the single most important architectural choice in the project, since hand-writing cleaning logic twice is the classic cause of "training/serving skew."

Group	Features	Imputation	Transform	Rationale
Numeric	age, trestbps, chol, thalach, oldpeak	Median	StandardScaler	Median is robust to the right-skew/outliers in Figure 3; sca
Categorical	sex, cp, fbs, restecg, exang, slope, ca, thal	Most frequent	OneHotEncoder (handle_unknown="ignore")	Coded categories, not ordinal scales (Figure 2); one-hot a

Both imputers fit only on the training split (scikit-learn's Pipeline semantics enforce this automatically), preventing data leakage. Critically, this transformer is not a standalone script — it is wrapped inside the same `Pipeline` object as the classifier and serialized together with it (Section 8), so the FastAPI service never re-implements feature engineering; it loads one artifact and the identical imputation/scaling/encoding applies automatically at inference time.

6. Model Development & Hyperparameter Tuning

Two model families are trained and compared in `src/train.py`: **Logistic Regression** (linear, interpretable, low-variance) and **Random Forest** (non-linear ensemble), chosen to test whether this problem genuinely benefits from non-linearity. The data is split 80/20 with a fixed seed (`random_state=42`) and **stratified** sampling so the 54/46 class ratio is preserved; hyperparameter search uses 5-fold stratified cross-validation, which matters given only ~242 training rows. Both are tuned with `GridSearchCV` optimizing **ROC-AUC** (threshold-independent ranking quality) rather than accuracy, keeping the deployed decision threshold a separate, later choice.

Model	Hyperparameters searched	Best values found
Logistic Regression	C ∈ {0.01, 0.1, 1.0, 10.0}	C = 1.0
Random Forest	n_estimators ∈ {100,200,400}; max_depth ∈ {None,4,8}; min_samples_leaf ∈ {1,2,4}	n_estimators=400, max_depth=4, min_samples_leaf=4

Random Forest converging on a shallow, heavily-regularized tree (depth 4) rather than an unconstrained forest is itself informative: on a dataset this small, more flexibility tends to overfit the training folds — a pattern consistent with the final result below.

7. Model Evaluation and Results

Both tuned models were evaluated on the same held-out 20% test split (61 patients).

Model	CV ROC-AUC	Test Acc.	Test Prec.	Test Recall	Test F1	Test ROC-AUC
Logistic Regression	0.902	0.885	0.839	0.929	0.881	0.966 (best)
Random Forest	0.902	0.852	0.806	0.893	0.847	0.948

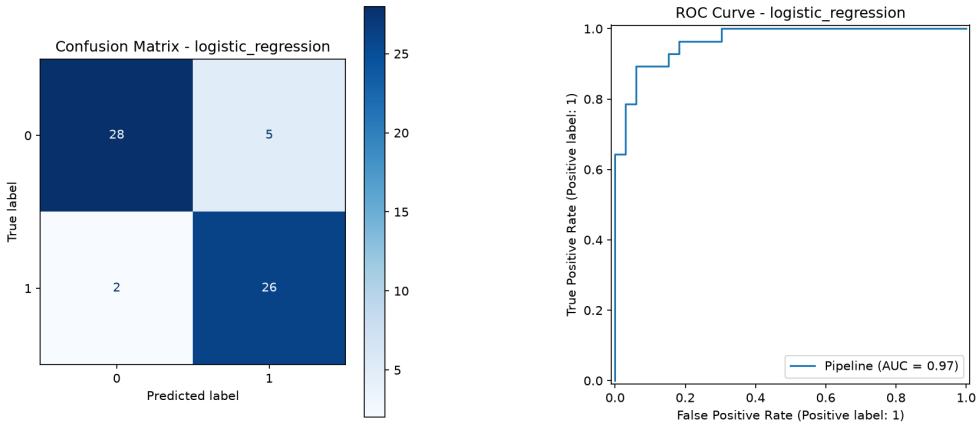


Figure 6. Confusion matrix and ROC curve for the selected Logistic Regression model (test set).

The two models are tied on cross-validated ROC-AUC (0.902 each) but Logistic Regression pulls ahead on every test-set metric — the classic signature of the more flexible Random Forest having fit slightly more to training-fold idiosyncrasies. **Logistic Regression was selected as the production model.** Recall was weighted heavily in this choice given the clinical framing: a missed disease-positive patient (false negative) is costlier than an unnecessary follow-up test (false positive), and Logistic Regression's 92.9% recall misses only ~7% of positives, vs. 10.7% for Random Forest. It is also simpler, faster to serve, and directly interpretable via its coefficients.

8. Experiment Tracking with MLflow & Model Packaging

Every call to `src/train.py` creates one MLflow run per model under the experiment `heart-disease-risk-classification`, logging:

- **Parameters** — the winning hyperparameters selected by GridSearchCV for that model (e.g. `best_model__C = 1.0` for Logistic Regression).
- **Metrics** — cross-validated ROC-AUC, plus test accuracy, precision, recall, F1, and ROC-AUC, logged as named, queryable numeric metrics rather than buried in a log file.
- **Artifacts** — the confusion matrix plot, the ROC curve plot, and the fitted sklearn pipeline itself, via `mlflow.sklearn.log_model()`.

A practical issue encountered: MLflow 3.x marked its original plain filesystem tracking backend (the classic `./mlruns` directory) as deprecated, placing it into "maintenance mode" and raising an

exception by default. This project therefore uses a local SQLite database (`sqlite:///mlflow.db`) as the tracking-store backend instead, the currently recommended lightweight option for a single-machine setup; run artifacts (plots and model files) are still stored on the local filesystem under `mlruns/`, since MLflow separates the metadata store from the artifact store. To inspect runs interactively: `mlflow ui --backend-store-uri sqlite:///mlflow.db`, then open `http://127.0.0.1:5000`.

The winning pipeline (preprocessing + estimator, as one object) is separately serialized with `joblib` to `models/model.pkl` — chosen over plain `pickle` for its efficiency with the large NumPy arrays inside a fitted scikit-learn estimator (the one-hot encoder's category lists, the scaler's mean/variance vectors, the regression coefficients), and it is what the FastAPI service actually loads at startup. Reproducibility rests on four pillars, each exercised automatically by CI: pinned dependencies in `requirements.txt`; a scripted, never-manually-run data download; one single preprocessing+model artifact with no separate script that could drift out of sync; and a fixed random seed (`random_state=42`) used consistently for the train/test split, the cross-validation folds, and both estimators.

9. Automated Testing Strategy

14 pytest tests across three files, each targeting a distinct layer, with a deliberate constraint: tests that run earliest in CI must never depend on network access or a trained model, so a broken environment — not a flaky dependency — is always the true cause of a CI failure.

- **test_data_processing.py** (6 tests, synthetic in-memory data) — duplicate removal, column selection, a clear error on a missing required column, correct feature/target split, no NaNs surviving imputation, and no crash on a category never seen during fitting.
- **test_model.py** (3 tests, `make_classification` synthetic data) — the pipeline trains and beats a random baseline; predicted probabilities are well-formed; and a joblib save/reload round-trip reproduces bit-for-bit identical predictions.
- **test_api.py** (5 tests, real FastAPI `TestClient`, auto-skipped if no trained model exists) — `/health` reports the model loaded; a valid `/predict` call returns a well-formed response; out-of-range or missing fields are rejected with HTTP 422 by Pydantic before reaching the model; and `/metrics` is exposed.

10. CI/CD Pipeline Design (GitHub Actions)

`.github/workflows/ci.yml` defines four sequential, gated jobs, so a failure at any stage stops the pipeline immediately with an isolated, clear error.

Stage	What it does	Why gated separately
1. Lint	flake8 + black --check across all Python source directories.	Fails fast on trivial issues before spending CI minutes on training/Docker.
2. Unit Tests	Offline, synthetic-data tests only, with coverage, uploaded as an artifact.	Validates code correctness independent of network access or a trained model.
3. Train	Downloads real data, runs EDA, trains + logs to MLflow, runs API tests against the freshly trained model; uploads model.pkl + reports/ as artifacts.	Separates "code works" from "model trains end-to-end on real data," and produces the artifact stage 4 needs.
4. Docker Build & Smoke Test	Downloads the model artifact, builds the image, runs the container, polls /health, calls /predict with sample data.	Concretely proves the model serves correctly in an isolated container, not just that the image builds.

Every stage dumps clear logs on failure (Docker logs are printed with `if: always()`), satisfying the requirement that "the pipeline must fail on code or test errors and give clear logs." This was validated first-hand during development: a formatting violation in a helper script, added after the last local lint check, was caught by the Lint job before it could reach the main branch in a broken state (see Section 16).

11. Model Containerization with Docker

The service is FastAPI (`api/main.py`), chosen over Flask for built-in Pydantic request/response validation, auto-generated OpenAPI docs at `/docs`, and native async support.

Endpoint	Purpose
GET /health	Liveness/readiness probe target; reports whether the model is actually loaded, not just whether the proc
POST /predict	Accepts 13 clinical features as JSON; returns a 0/1 prediction, label, confidence, and model class name.
GET /metrics	Prometheus-format scrape target (request counts, latency histograms), via prometheus-fastapi-instrume

The `Dockerfile` builds from `python:3.11-slim`, copies only `src/`, `api/`, and the trained `models/` (excluding tests, data, and docs via `.dockerignore`), runs as a non-root user, and declares a `HEALTHCHECK` polling `/health` every 30s.

```
python src/train.py && docker build -t heart-disease-api:local .
docker run -p 8000:8000 heart-disease-api:local
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @sample_reques
t.json
```

```
# -> {"prediction": 0, "label": "no_disease", "confidence": 0.8146, "model_name": "LogisticRegre
ssion"}
```

12. Production Deployment to Kubernetes

`k8s/deployment.yaml` runs **2 replicas** with readiness and liveness probes on `/health` (so a still-starting or broken pod is automatically removed from rotation, or restarted) and CPU/memory requests and limits. `k8s/service.yaml` is a `LoadBalancer` Service, which works out of the box with Docker Desktop Kubernetes and Minikube's `tunnel` command without the added complexity of an Ingress controller for a single-service deployment.

```
kubectl apply -f k8s/deployment.yaml && kubectl apply -f k8s/service.yaml
kubectl get pods && kubectl get svc heart-disease-api
kubectl port-forward svc/heart-disease-api 8080:80
curl http://localhost:8080/health
```

Kubernetes' self-healing behavior can also be demonstrated directly: scaling the deployment to four replicas (`kubectl scale deployment heart-disease-api --replicas=4`) and then deliberately deleting one pod shows Kubernetes automatically scheduling a replacement to restore the desired replica count, with no manual intervention.

On deployment environment choice: this project was developed on a corporate-managed Windows laptop, where Docker Desktop's licensing requires a paid subscription at large organizations and the Windows Package Manager CLI was found explicitly disabled by Group Policy. To keep deployment reproducible regardless of a reviewer's machine, `k8s/README.md` documents four equally valid setups — Docker Desktop, Minikube on Docker, Minikube on Hyper-V (no Docker Desktop), and **GitHub Codespaces** (pre-configured via `.devcontainer/devcontainer.json` with Docker-in-Docker and kubectl/Minikube, needing zero local install) — the last of which was used to capture this submission's deployment evidence.

13. Monitoring and Logging

A FastAPI middleware logs every request as a structured JSON line (method, path, status code, and latency in milliseconds; `/predict` calls additionally log the prediction and confidence) to both stdout, which is captured automatically by `docker logs` / `kubectl logs` with zero extra configuration, and to a local `logs/api.log` file for convenience during development. `prometheus-fastapi-instrumentator` automatically instruments the application and exposes `/metrics` in the standard Prometheus exposition format — request counts by endpoint and status code, latency histograms, and an in-progress-requests gauge — with no manual metric-instrumentation code anywhere in the application. `docker-compose.yml` runs the API alongside a Prometheus instance (`monitoring/prometheus.yml` scrapes `/metrics` every 10 seconds) and a Grafana instance, so a request-rate and latency dashboard can be built against continuously-updating live data with a single `docker compose up -d`, reachable at `localhost:9090` (Prometheus) and `localhost:3000` (Grafana, default login admin/admin).

14. System Architecture

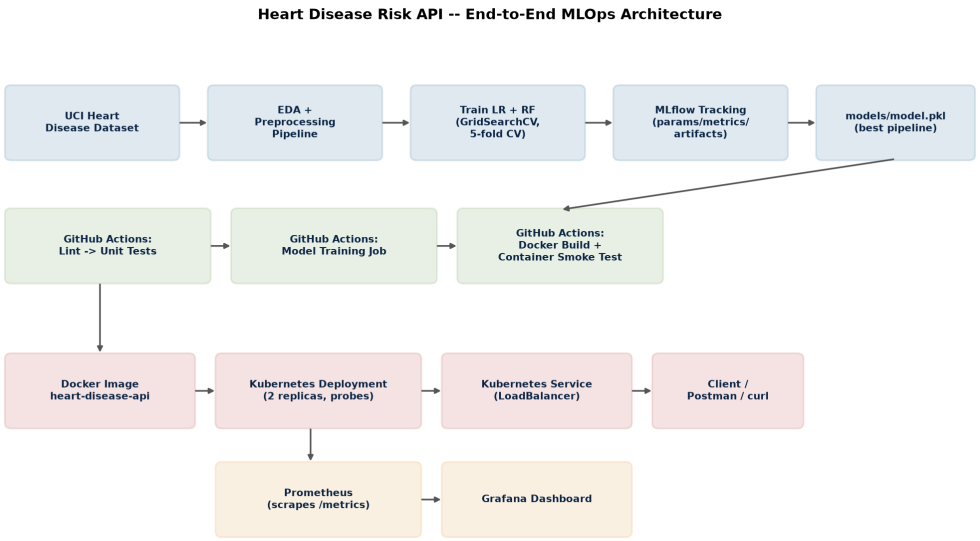


Figure 7. End-to-end MLOps architecture, from raw data to a monitored, deployed service.

The diagram traces the complete lifecycle of a model version through the system. Data flows from the UCI source through EDA and the preprocessing pipeline into model training, which is tracked by MLflow and produces the single `models/model.pkl` artifact (Section 8). GitHub Actions (Section 10) orchestrates linting, testing, training, and a Docker build-and-smoke-test in strict sequence, producing a verified image handed off to Kubernetes (Section 12), which runs it behind a load-balanced Service with automatic health checking and self-healing. Finally, Prometheus continuously scrapes the running pods' `/metrics` endpoint and feeds a Grafana dashboard, closing the loop from "a model was trained" to "the model's real-world serving behavior is observable."

15. Key Design Decisions

This section consolidates, in one place, the principal technical decisions made throughout the project and the reasoning behind each one, so the overall design logic of the system is easy to review at a glance rather than having to be reconstructed from scattered justifications across earlier sections.

Decision	Why
UCI official source, not a pre-cleaned mirror	Matches the brief exactly; demonstrates raw-data handling; reduces overlap with other subm
ColumnTransformer inside the saved Pipeline	Eliminates train/serve skew by construction — one artifact, fit once, used everywhere.
Logistic Regression over Random Forest	Better test-set generalization (accuracy, recall, F1, ROC-AUC), simpler, interpretable.
SQLite MLflow backend	The filesystem backend is in maintenance mode as of MLflow 3.x.
Offline synthetic-data unit tests	Keeps lint/unit-test CI fast and independent of network access or a pre-trained model.
FastAPI over Flask	Built-in Pydantic validation, auto OpenAPI docs, native async.
LoadBalancer Service type	Works out of the box with Docker Desktop/minikube tunnel, no Ingress needed for one servi
GitHub Codespaces as a deployment option	Avoids Docker Desktop's enterprise licensing/IT-policy restrictions entirely, zero local install

16. Challenges Faced and Resolutions

- **MLflow filesystem backend deprecation** — the first training run raised `MlflowException` on startup; resolved by switching to a SQLite tracking URI.
- **Unsafe-deserialization guard on model logging** — MLflow 3.x's default skops format rejected a `numpy.dtype` inside the pipeline; resolved by requesting `serialization_format="cloudpickle"` for the MLflow-logged copy.
- **A flake8 regression caught by CI after pushing to GitHub** — a helper script written after the last local lint pass had an indentation violation; the Lint job correctly failed the pipeline, it was fixed in under a minute by re-running flake8 locally, and pushed as a follow-up commit — a first-hand demonstration of CI/CD catching a real regression before it could cause harm downstream.
- **Docker Desktop unavailable on the development machine** — treated as a real deployment-environment constraint rather than a blocker, leading directly to the four-option deployment documentation in Section 12.

17. Conclusion, Future Work & Repository

This project delivers a complete, reproducible MLOps pipeline: UCI-sourced data acquisition, documented EDA, two compared models with Logistic Regression selected on test-set generalization grounds, full MLflow tracking, a single reproducible model artifact, a 14-test suite, a four-stage fail-fast CI/CD pipeline with a real Docker smoke test, four documented Kubernetes deployment

paths, and a Prometheus/Grafana monitoring stack. Natural extensions include registering the model in MLflow's Model Registry with formal staging/production transitions, automated data-drift monitoring in production, extending CI into full continuous deployment (auto-push to a registry and rolling Kubernetes updates on a successful main-branch build), and adding explainability tooling (e.g. SHAP values) to the API response for clinical interpretability of individual predictions.

Code repository: https://github.com/SaiSowjanyaVadapalli/heart-disease-mlops

Top-level project layout:

```
heart-disease-mlops/
|-- scripts/download_data.py      # UCI data acquisition
|-- notebooks/01_eda.py           # EDA -> reports/figures/, EDA_SUMMARY.md
|-- src/
|   |-- data_processing.py        # cleaning + preprocessing ColumnTransformer
|   |-- train.py                  # trains LR + RF, MLflow tracking, saves best model
|   `-- predict.py                # shared inference helper (API + tests)
|-- api/
|   |-- main.py                   # FastAPI app: /predict, /health, /metrics
|   `-- schemas.py                # Pydantic request/response models
|-- tests/                        # 14 pytest unit + integration tests
|-- models/model.pkl              # trained pipeline artifact (generated)
|-- reports/                      # EDA figures, metrics.json (generated)
|-- Dockerfile, docker-compose.yml
|-- k8s/                          # Deployment + Service manifests, 4 setup guides
|-- monitoring/prometheus.yml
|-- .devcontainer/                # GitHub Codespaces config (Docker-in-Docker, kubectl)
`-- .github/workflows/ci.yml      # lint -> unit test -> train -> docker build/smoke-test
```

Roll Number: 2025AE05681 | **Course:** AIMLCZG523 MLOps | **Assignment 1**